

universidad de león Online

Universidad de León ▪ Máster Universitario en Ciberseguridad

# Ingeniería del software seguro en la era de la IA generativa

## Diseño y Programación Seguras

---

## Aislamiento técnico y riesgos de los agentes

Francisco Javier Rodríguez Lera

Material elaborado con asistencia de IA generativa (DeepSeek-V4-Flash; considerados también GPT-6 y Claude Sonnet 4.6). Contenido revisado y validado por el autor.



Instituto de Investigación e Innovación en Ingeniería

Escuela de Ingenierías Industrial, Informática y Aeroespacial

universidad de León

# Objetivos de aprendizaje

Al terminar el capítulo, el estudiante será capaz de:

- **Distinguir** el riesgo del código producido del riesgo de las acciones del agente.
- **Construir** un modelo de amenazas con límites de confianza.
- **Enumerar** las superficies de acción de un agente.
- **Comparar** procesos restringidos, contenedores, VMs y microVMs.
- **Justificar** controles de privilegios mínimos y confinamiento.

# Dos objetos de riesgo: el código y las acciones

- El problema cambia de naturaleza cuando el agente pasa de generar texto a *actuar*: ejecutar comandos, escribir ficheros, instalar paquetes, leer secretos.
- **Código producido:** el artefacto escrito; pregunta guía «¿qué propiedades tiene el artefacto?». Riesgos del software en general (validación, deserialización, *path traversal*, *TOCTOU*, secretos).
- **Acciones del agente:** lo que hace con las herramientas; pregunta guía «¿qué puede hacer, sobre qué recursos, con qué autoridad y con qué consecuencias?».
- Un agente que nunca escribió una línea vulnerable puede exfiltrar un secreto o borrar un directorio.

# Dos objetos de riesgo

| Dimensión | Código producido | Acciones del agente |
|---|---|---|
| Qué es | El artefacto escrito | El comportamiento mientras trabaja |
| Pregunta guía | ¿Qué propiedades tiene el artefacto? | ¿Qué puede hacer y con qué autoridad? |
| Momento | Después de generar | Durante el desarrollo |
| Ejemplo (caso) | Fórmula no escapada; ruta no canónica | Abrir red o instalar un paquete |
| Marco de análisis | Familias CWE (22, 367, 502, 798, 400) | Modelo de amenazas y límites de confianza |
| Control típico | Revisión, SAST, pruebas, validación | Aislamiento, privilegios, auditoría, aprobaciones |
| Límite | Pruebas finitas no demuestran seguridad | El aislamiento reduce el daño; no certifica |

La distinción es **Evidencia**: objetos de análisis diferentes, con marcos y controles diferentes.

# Modelo de amenazas y límites de confianza

- Un modelo de amenazas es una declaración explícita y acotada de qué se protege, de quién, con qué capacidades y bajo qué supuestos.
- **No** es una lista de vulnerabilidades: es el encuadre que permite decidir qué controles son pertinentes.
- Su valor docente: obliga a escribir los supuestos y a hacer visibles los que se dan por ciertos.

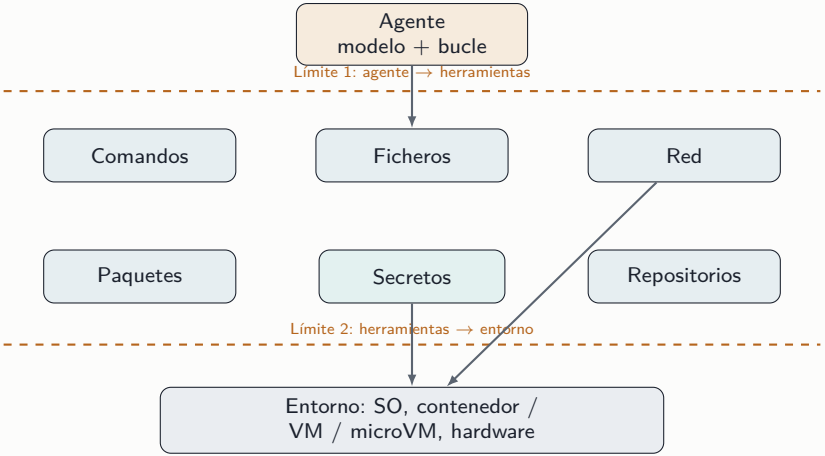
# Activos, adversario y límites de confianza

**Activos.** Integridad del repositorio; confidencialidad de secretos; integridad del anfitrión; disponibilidad del servicio; integridad de la cadena de suministro.

**Adversario.** No solo un atacante externo: también el *contenido no fiable* que el agente procesa (CSV de un tercero, README, web) con instrucciones hostiles.

**Límites de confianza.** Tres zonas de confianza decreciente: el *agente*, las *herramientas* (actúan sobre recursos reales) y el *entorno* (anfitrión y red).

# Límite de confianza entre agente, herramientas y entorno



# Modelo de amenazas del flujo agente-herramientas-entorno

| Activo | Amenaza | Superficie | Control candidato | Etiqueta |
|---|---|---|---|---|
| Repositorio | Escritura no autorizada | Herramienta de ficheros | Rama aislada; revisión; aprobación | Evidencia |
| Secretos | Exfiltración | Variables de entorno; red | Minimización; red restringida | Propuesta |
| Anfitrión | Escape del confinamiento | Kernel; montajes | VM o microVM; sin montajes de escritura | Hipótesis |
| Disponibilidad | Agotamiento de recursos | CPU, memoria, disco | Cuotas; límite de tiempo | Propuesta |
| Cadena de suministro | Dependencia comprometida | Gestor de paquetes | Réplicas internas; verificación | Hipótesis |
| Decisiones | *Prompt injection* | Contenido no fiable | Tratar el contenido como dato | Propuesta |

# ¿Es «el mismo sistema» el que genera y el que actúa?

- Que el agente sea el mismo en ambos casos, ¿implica que las dos preguntas se respondan igual?

# Superficies de acción del agente

- Las herramientas que el agente puede invocar definen su *superficie de acción*.
- **Comandos del sistema:** la frontera entre «ejecutar la herramienta» y «ejecutar el contenido no fiable» es permeable (vector de inyección).
- **Lectura/escritura de ficheros:** leer permite exfiltrar; escribir permite destruir o introducir código. Salidas confinadas a un directorio efímero.
- **Red:** canal natural de exfiltración y de descarga; por defecto, sin red.
- **Paquetes y dependencias:** decisión de cadena de suministro (origen, versión, integridad).

# Secretos, repositorios y prompt injection

**Secretos.** El agente *no debe ver* más credenciales de las necesarias; las que ve, de corta duración y alcance mínimo.

**Repositorios.** Escribir tiene efectos persistentes y compartidos: el agente puede preparar un cambio en una rama aislada, pero la incorporación exige aprobación humana explícita.

**Prompt injection.** El contenido no fiable (CSV, README, web) puede incluir instrucciones dirigidas [2, 1, 6]. La defensa no es una lista de palabras prohibidas, sino *no dar autoridad al contenido no fiable*: se trata como dato.

**Cadena de suministro.** Origen del modelo, herramientas, dependencias y artefactos; su integridad se apoya en procedencia y reproducibilidad, no en confianza por defecto.

**La superficie de acción es una decisión de diseño:** cada herramienta habilitada debe corresponder a una necesidad de la tarea (privilegios mínimos).

# Procesos restringidos, contenedores, VMs y microVMs

- No todos los mecanismos ofrecen las mismas garantías; el límite crece de menor a mayor.
- **Proceso restringido:** syscalls acotadas, MAC, límites de recursos, FS reducido; ligero, pero comparte el núcleo.
- **Contenedor:** empaqueta proceso + FS + namespaces; comparte el núcleo. Aislamiento de recursos y organización, *no* frontera de seguridad frente a código hostil.
- **VM:** núcleo propio sobre hipervisor; límite más fuerte, mayor coste.
- **MicroVM:** límite comparable a la VM con menor arranque y huella; cargas efímeras.

# Comparación de mecanismos de aislamiento

| Mecanismo | Núcleo | Fortaleza del límite | Coste / uso | Límite |
|---|---|---|---|---|
| Proceso restringido | Compartido | Baja-media | Muy bajo; tareas acotadas | Fallo de política o núcleo |
| Contenedor | Compartido | Media si reforzado; baja si no | Bajo; reproducibilidad | No es frontera de seguridad |
| VM | Propio | Alta (hipervisor) | Alto; aislamiento fuerte | Superficie del hipervisor |
| MicroVM | Propio | Alta (hipervisor) | Medio; cargas efímeras | Superficie del hipervisor |

**Un contenedor ordinario no es un límite de seguridad:** comparte el núcleo. Sirve para empaquetar, reproducir y separar recursos, no para demostrar contención de código hostil.

## «El contenedor hace que el código sea seguro»

- ¿Qué error conceptual encierra esa afirmación y qué habría que verificar?

# Controles propuestos: privilegios mínimos y confinamiento

- **Privilegios mínimos:** usuario no privilegiado, sin capacidades elevadas, sin escritura fuera del directorio de trabajo, sin red salvo excepción, credenciales mínimas y de corta duración. *El privilegio que no se concede no puede abusarse.*
- **Aislamiento de red:** por defecto sin red; destinos declarados y registrados.
- **FS efímero:** solo lectura en lo posible, directorio de trabajo aislado; se descarta al terminar.
- **Límites de CPU, memoria y tiempo:** acotan bucles y asignaciones desmedidas.
- **Control de secretos:** no montar los que no se necesiten; alcance mínimo y corta duración.
- **Auditoría:** registrar acciones, identidad y resultado (base de la trazabilidad).
- **Aprobaciones:** puerta humana para acciones irreversibles; el agente *propone*, el humano *decide*.
- **Limpieza:** destruir entorno y recursos temporales al terminar.

# Los controles no se suman como garantías

- Aplicar todos los controles reduce la superficie y acota el daño, pero *no demuestra* que el aislamiento sea correcto frente a cualquier código hostil.
- La suficiencia de una configuración concreta es una **Hipótesis** que debe probarse y auditarse para *ese* entorno y *esas* capacidades.
- NIST SSDF [4], ASVS [5] e ISO/IEC 27034 [3] son marcos de referencia; ninguno certifica un sandbox para agentes.

# Clases de ataque (análisis defensivo, no ejecución)

| Clase | Defensa conceptual |
|---|---|
| A1 - Instrucción embebida en contenido | No dar autoridad a datos no fiables; aprobación humana; verificar origen |
| A2 - Lectura de secreto del entorno | Minimizar secretos y su alcance; revisar la salida |
| A3 - Dependencia no verificada | Fijar versiones; réplicas internas; verificar integridad |
| A4 - Agotamiento de recursos | Cuotas de recursos y límite de tiempo; terminación controlada |
| A5 - Escritura no autorizada en repositorio | Rama aislada, revisión y aprobación; auditoría |
| A6 - Escape del confinamiento | Mecanismo proporcional (VM/microVM); endurecer; verificar |

Material para análisis, *no* para ejecución fuera de aislamiento autorizado.

## Síntesis del capítulo

- El riesgo del **código** y el de las **acciones** del agente son objetos distintos, con marcos y controles distintos.
- El modelo de amenazas declara activos, adversario y límites de confianza; el contenido no fiable es adversario.
- La superficie de acción es una decisión de diseño: privilegios mínimos.
- El aislamiento crece de proceso restringido a contenedor, VM y microVM; un contenedor ordinario *no* es un límite de seguridad.
- Los controles reducen el riesgo pero no demuestran suficiencia: es una **Hipótesis**.
- Toda afirmación «ejecutarlo en un contenedor es seguro» debe leerse como «se aplicó un aislamiento cuya suficiencia no se ha verificado».

# Preguntas de reflexión

1. ¿Por qué el riesgo del código y el de las acciones exigen marcos distintos?
2. Enumerad los activos de una sesión CSV→PDF/XLSX y asignad un límite de confianza.
3. ¿Qué error conceptual comete quien dice «el contenedor hace seguro el código»?
4. ¿Por qué la red es un privilegio y no una capacidad por defecto?
5. ¿Por qué el control de secretos no se resuelve solo ocultando el secreto?

# ¿Basta con «un contenedor» para código no fiable?

- Ejecutar código generado por el asistente, ¿exige más que un contenedor?

# Referencias I




Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr.

Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents.

In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

Publicación científica.



Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz.

Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection, 2023.

Prepublicación; evidencia de vectores de inyección indirecta.



ISO/IEC.

Iso/iec 27034 — seguridad de la información: Seguridad de las aplicaciones.

Norma; debe citarse la parte y edición específicas al aplicar requisitos.

# Referencias II




National Institute of Standards and Technology.

Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities.

Technical Report SP 800-218, NIST, 2022.

Norma / marco de referencia.



OWASP Foundation.

Application security verification standard (asvs), 2025.

Guía de referencia (versión estable 5.0.0, 2025).



OWASP GenAI Security Project.

Owasp top 10 for llm applications 2025, 2025.

Guía de referencia; no confundir con OWASP Top 10 para aplicaciones web.